Guía del notebook `1_11_retropropagacion_completo`: entrenar una red multicapa

Puntos: 3 · **Tiempo:** 30–40 min · **Módulo:** 4, redes multicapa (clase 5) · **Archivo:** `notebooks/1_11_retropropagacion_completo.ipynb`
Requisito: haber hecho `1_07` (entrenar una neurona) y `1_10` (un paso de retropropagación).

¿Qué vas a aprender?

- A juntar todo: pase hacia adelante, retropropagación y descenso por gradiente, durante muchas épocas.
- A dibujar la **curva de aprendizaje** e interpretarla.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_11_retropropagacion_completo.ipynb` → kernel **nni**.
   El notebook usa `data_prep.py` y el paquete `nni` de la carpeta `notebooks`.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Mismos datos que en `1_07`:** admisiones a un posgrado, 6 entradas, 360 alumnos para entrenar y 40 para probar.
- **Red:** 6 entradas → **2 neuronas ocultas** → 1 salida, todas con sigmoide.
- **Cada época:**
  1. Pone los incrementos acumulados en cero.
  2. Para cada alumno: pase hacia adelante, error, término de error de la salida y de la oculta (igual que en `1_10`), y **acumula** los incrementos.
  3. Al terminar los alumnos, **actualiza** las dos matrices de pesos.

Paso a paso

Celdas 1–2: datos

- Estadísticas de las entradas: GRE y GPA ya están escalados (media ≈ 0, desviación ≈ 1).
- Clases: **243 no admitidos y 117 admitidos**.

Celda 3: tamaños e hiperparámetros

- `n_hidden = 2`, `epochs = 900` y `learnrate = 0.001`.
- Pesos iniciales al azar, normales con desviación 1/√6, y semilla 21.

Celda `TODO (1 punto)`: el ciclo de entrenamiento

Completa los huecos con lo que ya hiciste en `1_10`:

Hueco	Qué escribir
`hidden_input`	x·W_ih (`np.dot`)
`hidden_output`	sigmoide de lo anterior (`functions.sigmoid`)
`output`	sigmoide de hidden_output·W_ho
`error`	y − output
`output_error`	error · output · (1 − output)
`hidden_error`	output_error · `weights_hidden_output` · hidden_output · (1 − hidden_output)
`del_w_hidden_output +=`	output_error · hidden_output
`del_w_input_hidden +=`	hidden_error · x en forma de matriz (`x[:, None]`, como en `1_10`)
Actualizar `weights_... +=`	tasa · incremento acumulado

Dos opciones válidas para la actualización final:

- **A, sin dividir:** pesos += `learnrate` · acumulado, con `learnrate = 0.001`, como viene la plantilla.
- **B, promediado:** pesos += `learnrate` · acumulado / `n_records`, igual que en `1_07`. Así la tasa **no** depende del número de alumnos, pero hay que subirla a ≈ **0.5**.

✅ **Comprobación** (semilla 21, 900 épocas):

Opción	Pérdida inicial → final	Exactitud de prueba
A: 0.001 sin dividir	0.2646 → 0.2013	**0.725**
B: 0.5 con / n_records	0.2641 → 0.1999	**0.750**
⚠️ 0.005 con / n_records	0.2659 → 0.2516	0.425 (casi no aprende)

La última fila es una tasa **demasiado pequeña** para la opción B: con ella la red casi no aprende.

⚠️ **Detalle del notebook original:** el bloque que imprime `Train loss` calcula la pérdida usando **solo el último alumno** (`x`), no todos. Para una curva correcta, usa `features.values` en lugar de `x` en esas dos líneas.

Celda `TODO (1 punto)`: curva de aprendizaje

1. Antes del ciclo, crea una lista vacía, por ejemplo `historial = []`.
2. Al final de **cada** época, calcula la pérdida con todos los alumnos y agrégala con `.append`.
3. Grafica con `plt.plot(historial)`, poniendo nombres a los ejes (época y pérdida).

✅ **Comprobación:** la curva **baja rápido al principio y luego se aplana**.

Preguntas (1 punto)

- **¿La pérdida de entrenamiento disminuye?** Sí, de ≈ 0.26 a ≈ 0.20.

- **¿Cuál es la mejor pérdida que alcanzaste?** Anota el valor, las épocas y la tasa que usaste. Por ejemplo: "0.1999 con 900 épocas y tasa 0.5 promediada".

Para pensar

- **¿Por qué la exactitud (0.75) es casi igual a la de una sola neurona en `1_07` (0.775)?** Estos datos son casi linealmente separables, así que una capa oculta no ayuda mucho. Las capas ocultas brillan con problemas **no lineales**, como XOR o imágenes.
- **Reto:** sube `n_hidden` a 4 u 8. ¿Mejora la exactitud o aparece sobreajuste?

Errores frecuentes

Síntoma	Causa	Solución
`ValueError: operands could not be broadcast together with shapes (6,2) (6,)`	Sumaste `hidden_error * x` sin convertir x en columna	Usa `hidden_error * x[:, None]`
La pérdida no baja	Tasa muy pequeña para la opción B, o te faltó `+=`	Revisa la tabla de opciones
La pérdida impresa sube y baja raro	El cálculo original usa solo el último alumno	Usa `features.values`
`ModuleNotFoundError: data_prep`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
Tarda mucho	Son 900 épocas × 360 alumnos	Es normal: unos segundos. Ten paciencia o baja las épocas